Week 2 – Design & Initial Development

Project Area

Web & API Security

Project Title

APIShield – Automated API Security Analyzer

---

1. Introduction

During Week 2, the project moved from problem identification and research toward system design and initial development.

The objective was to design and begin implementing a lightweight API security analyzer capable of performing authorized security assessments against APIs in controlled environments.

The development focused on creating a secure foundation, an intentionally vulnerable test API, security validation utilities, a guarded HTTP client, security scanning logic, risk classification, database integration, and API endpoints for managing scans.

---

2. Proposed System Design

APIShield is designed as a modular application consisting of the following major components:

Frontend

Provides the user interface for initiating scans, viewing scan progress, reviewing findings, and accessing scan history.

Backend API

Built using FastAPI and responsible for handling scan requests, validation, scan management, and communication between application components.

Security Scanner

The scanner performs security checks against authorized API targets.

Safe HTTP Client

All outbound requests are handled through a controlled HTTP client that applies security restrictions such as SSRF protection, request timeouts, response-size limits, redirect validation, and request budgets.

Security Checks

The scanner contains checks related to:

- Authentication
- Authorization and access control
- Broken Object Level Authorization (BOLA)
- HTTP security headers
- CORS configuration
- Sensitive information exposure

Risk Engine

Findings are assigned severity and risk information using an explainable rule-based system.

Database

SQLite is used to store scan information, status, findings, and related assessment data.

---

3. Technology Stack

The initial implementation uses:

- **Python**
- **FastAPI**
- **SQLite**
- **httpx**
- **Pydantic**
- **pytest**
- **HTML/CSS/JavaScript**

The application is designed to run in a controlled and authorized environment.

---

4. Development Progress

Phase 1 – Foundation and Project Scaffold

The initial project structure was created with a secure application foundation.

Implemented components included:

- Application configuration
- Logging configuration
- FastAPI application structure
- API health endpoint
- Security-related middleware
- Error handling
- Standard API response structure
- Environment-based configuration
- Safe logging and credential masking
- Project documentation and configuration files

Security defaults were also established, including localhost-oriented operation, request timeouts, response-size limits, and request limits.

Phase 2 – Intentionally Vulnerable Test API

A local intentionally vulnerable FastAPI API was developed for testing APIShield.

The test API uses **fake data only** and contains controlled examples of API security weaknesses.

The test environment includes examples related to:

- Missing authentication
- Weak authentication
- Broken access control
- BOLA
- Sensitive information exposure
- Debug information exposure
- Insecure HTTP headers
- CORS misconfiguration
- Incorrect content types
- Redirect behavior

The vulnerable API provides a controlled ground-truth environment for validating whether APIShield can identify known security issues.

Phase 3 – Security Utilities

Security utilities were implemented to protect the scanner itself.

Important components include:

- URL validation
- SSRF protection
- IP address validation
- DNS resolution and validation
- Sensitive-value masking
- Evidence sanitization
- Rate limiting

These controls help prevent the scanner from being misused against unauthorized or unsafe targets.

Phase 4 – Safe HTTP Client and Scan Engine

A guarded HTTP communication layer was implemented.

The `SafeHttpClient` controls scanner requests and applies protections such as:

- SSRF checks before requests
- IP validation and pinning
- Redirect validation
- Request timeouts
- Response-size limits
- Request budgets
- Credential isolation
- Cookie isolation
- Unsafe state-changing request protection
- Proxy environment isolation

A basic scan engine and scan context were also introduced to provide a common framework for executing security checks.

Phase 5 – Security Checks and Risk Engine

The initial security detection framework was implemented.

Security checks were organized into modules covering:

- Information exposure
- HTTP/security headers
- Authentication
- Authorization

A rule-based risk engine was also introduced to provide:

- Finding IDs
- Severity levels
- Finding descriptions
- Severity rationale
- Risk scoring
- Overall assessment rating

The scanner was tested against the intentionally vulnerable API to verify that planted vulnerabilities were detected.

Phase 6 – Database, API Routes and Background Scans

The application was extended to support complete scan management.

Implemented functionality includes:

- SQLite database
- Scan creation
- Scan status tracking
- Scan result retrieval
- Scan deletion
- Dashboard statistics
- Background scan execution
- Scan progress tracking
- Database persistence

The scan API was also secured using input validation, SSRF pre-checks, authentication-data protection, request-size limits, rate limiting, and concurrent-scan controls.

---

5. Security-by-Design Measures

Security was considered during the design and implementation rather than being added only after development.

Important measures include:

1. **Authorized testing only**
    - The system is intended for authorized security assessments.
2. **SSRF protection**
    - Unsafe and private targets are blocked.
3. **Credential protection**
    - Sensitive authentication values are not stored as plain text in scan records.

4. **Evidence masking**
    - Sensitive values are masked before being stored or displayed.
5. **Request restrictions**
    - Timeouts, request budgets, and response-size limits help prevent resource abuse.
6. **Input validation**
    - Scan requests are strictly validated.
7. **Safe database operations**
    - Parameterized database operations are used.
8. **Controlled test environment**
    - The vulnerable API uses fake data and runs locally.

---

6. Initial Testing

Testing was performed continuously during development.

The project progressed through multiple test stages:

- Phase 1: 15 tests
- Phase 2: 34 total tests
- Phase 3: 209 total tests
- Phase 4: 298 total tests
- Phase 5: 419 total tests
- Phase 6: 509 total tests

The tests covered both normal functionality and security failure cases.

Mutation-style testing was also used during development to verify that important security controls could detect intentionally introduced implementation failures.

---

7. Current Development Status

At the end of the initial development stage, APIShield has progressed from a project scaffold into a functional API security assessment prototype.

The system can:

- Accept scan requests
- Validate scan targets
- Perform controlled HTTP requests
- Execute security checks
- Detect selected API security weaknesses
- Assign severity and risk information
- Store scan results

- Track scan progress
- Retrieve previous scan results
- Provide dashboard information

The intentionally vulnerable API provides a controlled environment for validating scanner accuracy.

---

8. Week 2 Outcome

The major outcome of Week 2 is the establishment of the core architecture and initial working implementation of APIShield.

The project now has a modular foundation consisting of:

API Request → Validation → Safe HTTP Client → Scan Engine → Security Checks → Risk Engine → Database → Results

This architecture provides a foundation for further development, testing, report generation, and usability improvements.

---

9. Next Development Direction

The next stage will focus on improving the reporting and usability aspects of the project.

Planned development includes:

- Security assessment report generation
- JSON report generation
- PDF report generation
- Report consistency and validation
- Improved dashboard/report presentation
- Additional testing and evaluation
- Documentation of limitations and results

---

10. Conclusion

Week 2 focused on converting the researched problem into a practical system design and initial working implementation.

APIShield now contains a secure project foundation, controlled vulnerable API, scanner security utilities, guarded HTTP communication, security detection modules, risk classification, database support, and scan management APIs.

The development follows a security-by-design approach and keeps testing within authorized and controlled environments.